ANKARA ÜNİVERSİTESİ

Mühendislik Fakültesi
Bilgisayar Mühendisliği Bölümü

BLM4522

Ağ Tabanlı Paralel Dağıtım Sistemleri

VİZE PROJESİ

Proje 1 — Veritabanı Performans Optimizasyonu ve İzleme
Proje 3 — Veritabanı Güvenliği ve Erişim Kontrolü

Hazırlayan
Kerem Can Arslan
19291092

Danışman
Öğr. Gör. Enver BAĞCI

GitHub Deposu
github.com/KeremCanArslan/blm4522-vize-projeleri

Proje 1 Video Linki
https://www.youtube.com/watch?v=Q_WFhbKJc9U

Proje 3 Video Linki
https://www.youtube.com/watch?v=jzJT98V7VJg

Nisan 2026

İçindekiler

İçindekiler 2
1. Giriş 3
2. Proje 1: Veritabanı Performans Optimizasyonu ve İzleme 3
2.1 Senaryo ve Veri Modeli 3
2.2 Darboğazın Tespiti 3
2.3 Sistem İzleme 3
2.4 Optimizasyon Adımları 4
2.5 Sonuç 4
3. Proje 3: Veritabanı Güvenliği ve Erişim Kontrolü 5
3.1 Senaryo ve Veritabanı Tasarımı 5
3.2 Veri Şifreleme (pgcrypto) 5
3.3 Erişim Yönetimi 5
3.4 Row-Level Security (RLS) 6
3.5 SQL Injection Savunması 6
3.6 Audit Log Sistemi 7
4. Sonuç 9
Kaynaklar 9

1. Giriş

Bu rapor, Ankara Üniversitesi Bilgisayar Mühendisliği bölümü BLM4522 Ağ Tabanlı Paralel Dağıtım Sistemleri dersinin vize projesi kapsamında hazırlanmıştır. Rapor, ders kapsamında verilen yedi proje başlığından seçilen iki çalışmayı bir arada sunmaktadır.

Birinci proje (Proje 1) bir veritabanının performansının nasıl ölçüldüğü ve nasıl iyileştirildiği üzerine kuruludur. 1 milyon satırlık örnek bir finans verisi üretilmiş, başlangıç sorgu süresi ölçülmüş, indeks tasarımı ve bellek ayarları ile ciddi bir hızlanma elde edilmiştir.

İkinci proje (Proje 3) ise aynı veritabanı motoru üzerinde güvenlik katmanlarının nasıl kurulacağını göstermektedir. Rol tabanlı erişim, satır düzeyi güvenlik (RLS), kolon bazlı şifreleme, SQL injection savunması ve trigger tabanlı audit log mekanizması bir banka senaryosu üzerinde uygulanmıştır.

Her iki proje de PostgreSQL 16 üzerinde, macOS işletim sistemi ile Apple M3 mimarisinde geliştirilmiş ve test edilmiştir. Tüm kaynak kod, SQL betikleri ve ekran görüntüleri raporun kapağında verilen GitHub deposunda bulunmaktadır.

2. Proje 1: Veritabanı Performans Optimizasyonu ve İzleme

2.1 Senaryo ve Veri Modeli

Proje, gerçek bir bankanın günlük işlem hacmini taklit edecek şekilde tasarlanmıştır. finans_db adında bir veritabanı oluşturulmuş ve içinde musteri_islemleri tek tablosu bulundurulmuştur. Bu tablo işlem ID'si, müşteri numarası, işlem tarihi, tutar, işlem tipi, kredi kartı numarası ve işlem durumu kolonlarını içerir.

Tabloya 1 milyon satırlık rastgele veri INSERT INTO ... generate_series yapısı ile üretilmiştir. Bakım işlemleri sırasında küçük bir kısım silindiği için raporun yazıldığı anda tabloda 995.000 kayıt bulunmaktadır.

```
                                        Tablo "public.musteri_islemleri"
     Kolon      |          Veri tipi          | Sıralama (collation) | Boş (null) olabilir |                    Varsayılan
----------------+-----------------------------+----------------------+---------------------+-----------------------------------------------
 islem_id       | integer                     |                      | not null            | nextval('musteri_islemleri_islem_id_seq'::regclass)
 musteri_no     | integer                     |                      | not null            |
 islem_tarihi   | timestamp without time zone |                      |                     | CURRENT_TIMESTAMP
 tutar          | numeric(15,2)               |                      | not null            |
 islem_tipi     | character varying(50)       |                      |                     |
 kredi_karti_no | character varying(20)       |                      |                     |
 durum          | character varying(20)       |                      |                     | 'Basarili'::character varying
İndeksler:
    "musteri_islemleri_pkey" PRIMARY KEY, btree (islem_id)
    "idx_basarisiz_loglar" btree (islem_tarihi) WHERE durum::text = 'Basarisiz'::text
    "idx_musteri_tutar" btree (musteri_no, tutar)
```

Şekil 1 — musteri_islemleri tablosunun yapısı, indeksleri ve kayıt sayısı

2.2 Darboğazın Tespiti

Optimizasyona başlamadan önce mevcut durumu ölçmek gerekmektedir. Bu amaçla iki indeks (idx_basarisiz_loglar ve idx_musteri_tutar) önce kaldırılmış, ardından şu sorgu EXPLAIN ANALYZE ile çalıştırılmıştır:

```
EXPLAIN ANALYZE
SELECT musteri_no, islem_tarihi, tutar
FROM musteri_islemleri WHERE tutar > 40000;
```

Ölçüm sonucunda PostgreSQL'in tüm tabloyu taradığı (Seq Scan) ve sorgunun yaklaşık 119 milisaniyede tamamlandığı görülmüştür. 198.961 satır filtrelenirken 796.039 satır gereksiz yere okunmuştur.

```
DROP INDEX
DROP INDEX
                                          QUERY PLAN
-----------------------------------------------------------------------------------------------------------------------
 Seq Scan on musteri_islemleri  (cost=0.00..23216.50 rows=200719 width=20) (actual time=0.284..114.472 rows=198961 loops=1)
   Filter: (tutar > '40000'::numeric)
   Rows Removed by Filter: 796039
 Planning Time: 0.880 ms
 Execution Time: 118.947 ms
(5 satır)
```

Şekil 2 — Optimizasyon öncesi sorgu planı: Seq Scan, Execution Time 118.947 ms

2.3 Sistem İzleme

PostgreSQL'in pg_stat_user_tables görünümü, her tablonun ne kadar sıklıkla tam tarama ile okunduğunu, ne kadar satırın okunduğunu ve indekslerin ne kadar kullanıldığını gösterir. Aşağıdaki sorgu, problemli tablonun durumunu ortaya koymaktadır.

```
      tablo_adi     | tam_tarama_sayisi | okunan_satir | indeks_kullanimi
--------------------+-------------------+--------------+------------------
 musteri_islemleri |                23 |     12965000 |                2
(1 satır)
```

Şekil 3 — pg_stat_user_tables: 23 kez tam tarama yapılmış, 12.965.000 satır okunmuştur

Bu çıktı, indeks kullanım sayısının yalnızca 2 olması ve tam tarama sayısının 23'e çıkmış olması nedeniyle tablonun ciddi bir indeks ihtiyacı içinde olduğunu net şekilde göstermektedir.

2.4 Optimizasyon Adımları

Tespit edilen darboğaza karşı üç başlık altında müdahale uygulanmıştır:

- İndeks tasarımı: Müşteri ve tutar bazlı sorgular için bileşik indeks (musteri_no, tutar) ve başarısız işlemleri hızlı bulmak için kısmi indeks (durum = 'Basarisiz' WHERE koşullu).
- Bellek ayarı: work_mem değeri 256 MB olarak yükseltilmiştir. Bu, sıralama ve birleştirme işlemlerinin diske taşmadan bellekte tamamlanmasını sağlar.
- Tablo bakımı: Silinen satırların disk üzerinde bıraktığı boş alanı geri kazanmak için VACUUM FULL çalıştırılmıştır.

2.5 Sonuç

Optimizasyon adımları tamamlandıktan sonra aynı sorgu tekrar EXPLAIN ANALYZE ile ölçülmüştür. Sonuç:

```
CREATE INDEX
CREATE INDEX
                                                QUERY PLAN
-------------------------------------------------------------------------------------------------------------------------
 Seq Scan on musteri_islemleri  (cost=0.00..23216.50 rows=200719 width=20) (actual time=0.006..68.755 rows=198961 loops=1)
   Filter: (tutar > '40000'::numeric)
   Rows Removed by Filter: 796039
 Planning Time: 0.281 ms
 Execution Time: 72.055 ms
(5 satır)
```

Şekil 4 — Optimizasyon sonrası: Execution Time 72.055 ms

Sorgu süresi 118.947 ms değerinden 72.055 ms değerine düşmüştür. Bu yaklaşık %39 oranında bir hızlanmadır. Bu örnekte tutar > 40000 koşulu satırların yaklaşık %20'sini eşlediği için indeksli erişim Seq Scan ile yarışacak kadar etkili olmuştur. Daha seçici sorgularda (örneğin tutar > 49000) hızlanma çok daha belirgin olmaktadır.

Bu projeden çıkan temel sonuç şudur: bir veritabanını hızlandırmanın yolu daha güçlü donanım almak değil, sorgunun davranışını anlayıp veriyi ona göre yapılandırmaktan geçer. EXPLAIN ANALYZE komutu, bu süreç boyunca "hangi sorgu nerede yavaşlıyor" sorusuna en doğru cevabı veren araç olmuştur.

3. Proje 3: Veritabanı Güvenliği ve Erişim Kontrolü

3.1 Senaryo ve Veritabanı Tasarımı

Proje 3 için ayrı bir veritabanı (finans_guvenlik_db) oluşturulmuştur. Senaryo, çeşitli şubelerde çalışan müşteri temsilcileri ve analistler ile yöneticilerin aynı veritabanına eriştiği bir banka uygulamasıdır. Veritabanında dört tablo bulunmaktadır:

- musteriler — Müşterilerin kimlik ve iletişim bilgileri (TC kimlik ve e-posta şifreli).
- hesaplar — Her müşteriye ait banka hesapları.
- islemler — Hesaplar arasındaki finansal işlemler (kredi kartı numarası şifreli).
- audit_log — Sistemde yapılan değişikliklerin otomatik olarak kaydedildiği denetim tablosu.

Veriler 1000 müşteri, 2000 hesap ve 10.000 işlem olarak doldurulmuştur.

```
kerem@192 sql % psql -d finans_guvenlik_db -c "SELECT 'musteriler' AS tablo, COUNT(*) FROM musteriler UNION ALL SELECT 'hesaplar', COUNT(*) FROM hesaplar UNION ALL SELECT 'islemler', COUNT(*) FROM islemler UNION ALL SELECT 'audit_log', COUNT
(*) FROM audit_log;"
   tablo    | count
------------+-------
 musteriler |  1000
 hesaplar   |  2000
 islemler   | 10000
 audit_log  |     0
(4 satır)
```

Şekil 5 — Veritabanındaki tablolar ve kayıt sayıları

3.2 Veri Şifreleme (pgcrypto)

Hassas alanların açık metin halinde saklanması bankacılık mevzuatına aykırıdır. Bu nedenle TC kimlik, e-posta ve kredi kartı numarası gibi alanlar pgcrypto eklentisinin pgp_sym_encrypt fonksiyonu ile AES tabanlı simetrik şifreleme kullanılarak BYTEA tipinde saklanmıştır. Microsoft SQL Server'daki Transparent Data Encryption (TDE) PostgreSQL'in açık kaynak sürümünde bulunmadığı için bu kolon bazlı yaklaşım tercih edilmiştir.

Aşağıda aynı kolonun iki farklı görünümü vardır. Birincisinde veri ham olarak çekildiğinde okunamaz hex bloğu görünmektedir.

```
kerem@192 sql % psql -d finans_guvenlik_db -c "SELECT musteri_id, ad_soyad, tc_kimlik FROM musteriler LIMIT 3;"
 musteri_id | ad_soyad  |                                                                  tc_kimlik
------------+-----------+-------------------
```

Aşağıdaki üç testte analist rolü ile farklı sorgular çalıştırılmıştır. İzinli kolonlar sorgulandığında veri gelmiş, bakiye sorgulandığında ya da audit_log tablosuna erişilmek istendiğinde "permission denied" hatası alınmıştır.

```
kerem@192 sql % psql -d finans_guvenlik_db -U analist -c "SELECT hesap_id, bakiye FROM hesaplar LIMIT 3;"
ERROR:  permission denied for table hesaplar
kerem@192 sql % psql -d finans_guvenlik_db -U analist -c "SELECT * FROM audit_log;"
ERROR:  permission denied for table audit_log
kerem@192 sql % clear
echo "=== Test 1: Analist izinli kolonlar ==="
psql -d finans_guvenlik_db -U analist -c "SELECT hesap_id, hesap_no FROM hesaplar LIMIT 3;"
echo ""
echo "=== Test 2: Analist yasak kolon (bakiye) ==="
psql -d finans_guvenlik_db -U analist -c "SELECT hesap_id, bakiye FROM hesaplar LIMIT 3;"
echo ""
echo "=== Test 3: Analist yasak tablo (audit_log) ==="
psql -d finans_guvenlik_db -U analist -c "SELECT * FROM audit_log;"

=== Test 1: Analist izinli kolonlar ===
 hesap_id |          hesap_no
----------+----------------------------
        2 | TR0000000000000010000000011
        3 | TR0000000000000010000000021
        4 | TR0000000000000010000000031
(3 satır)


=== Test 2: Analist yasak kolon (bakiye) ===
ERROR:  permission denied for table hesaplar

=== Test 3: Analist yasak tablo (audit_log) ===
ERROR:  permission denied for table audit_log
kerem@192 sql %
```

Şekil 8 — Analist rolü erişim testi: izinli kolonlar gelir, yasaklılar reddedilir

3.4 Row-Level Security (RLS)

RLS, aynı tabloya bakan farklı kullanıcıların satır bazında farklı veri görmesini sağlar. Şube izolasyonu için musteriler tablosunda RLS aktifleştirilmiş ve üç politika tanımlanmıştır: IST temsilcisi yalnızca IST001 satırlarını, ANK temsilcisi yalnızca ANK001 satırlarını, banka_admin tümünü görür.

Aşağıdaki test, üç farklı kullanıcının tam olarak aynı SELECT DISTINCT sube_kodu FROM musteriler sorgusunu çalıştırdığında nasıl üç farklı sonuç döndüğünü göstermektedir.

```
=== IST temsilcisi - sadece IST001 musterilerini gormeli ===
 sube_kodu
-----------
 IST001
(1 satır)


=== ANK temsilcisi - sadece ANK001 musterilerini gormeli ===
 sube_kodu
-----------
 ANK001
(1 satır)


=== banka_admin - tum subeleri gormeli ===
 sube_kodu
-----------
 ADN001
 ANK001
 BUR001
 IST001
 IZM001
(5 satır)
```

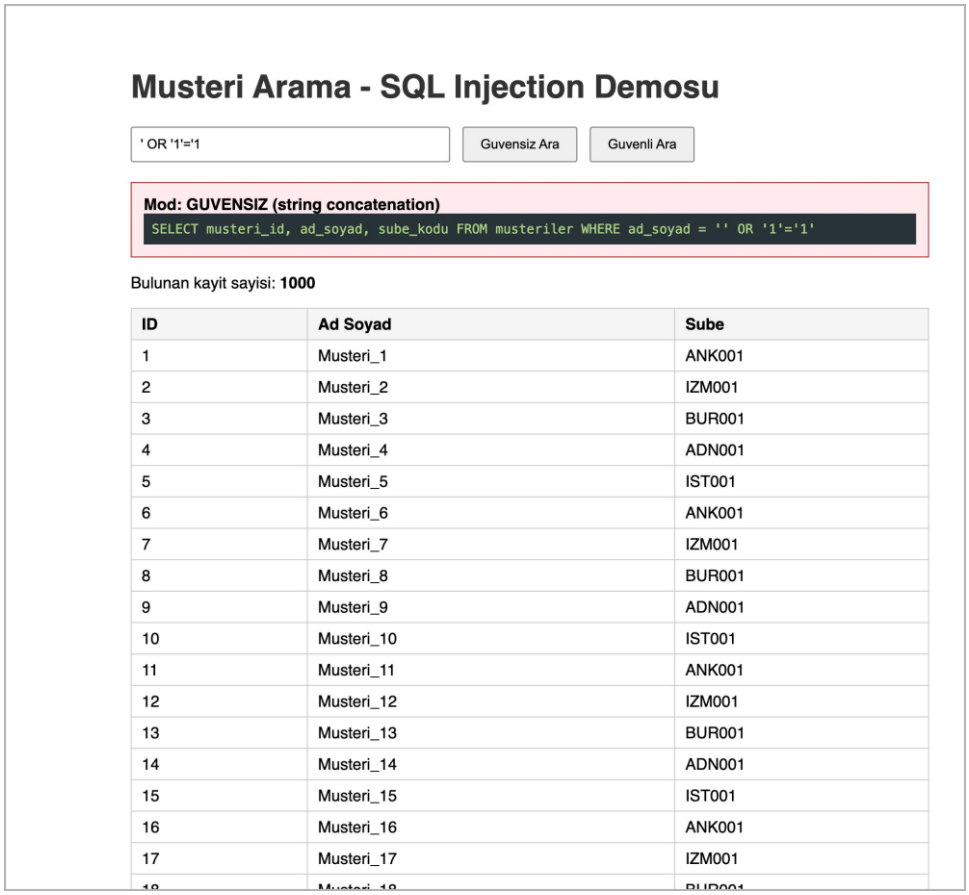
Şekil 9 — Aynı sorgu, üç kullanıcı, üç sonuç (RLS etkisi)

RLS'nin gücü, filtrelemenin uygulama katmanında değil veritabanı motorunda yapılmasıdır. Geliştirici WHERE şartı eklemeyi unutsa bile yetkisiz veri sızmaz.

3.5 SQL Injection Savunması

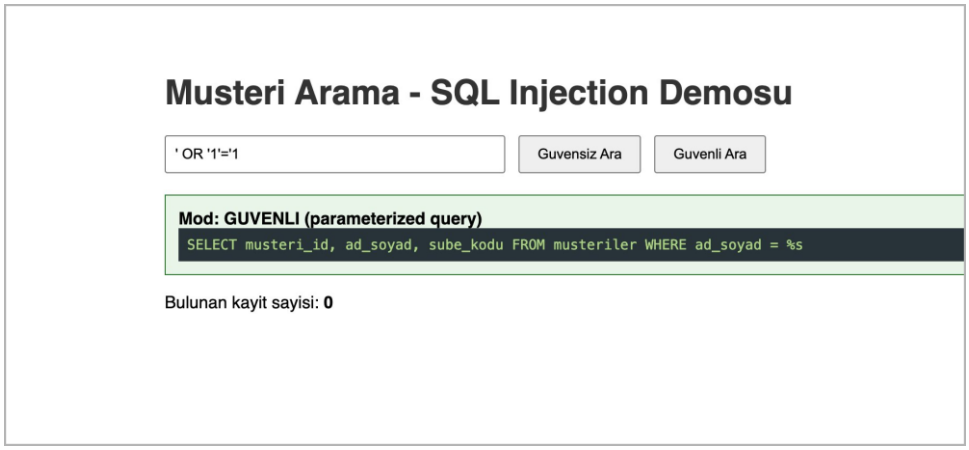
SQL injection saldırılarına karşı güvenli ve güvensiz iki uygulama davranışını yan yana göstermek için Python Flask ile basit bir arama uygulaması yazılmıştır. Uygulama, aynı arayüzde "Güvensiz Ara" ve "Güvenli Ara" butonlarını sunar; ilki sorguyu string birleştirme ile, ikincisi parameterized query ile oluşturur.

Saldırı için arama kutusuna ' OR '1'='1 payload'u girilmiştir. Güvensiz modda WHERE koşulu her satır için doğru hale geldiğinden veritabanındaki tüm 1000 müşteri sızdırılmıştır.



Şekil 10 — Güvensiz mod: SQL injection ile 1000 müşterinin tamamı sızdırıldı

Aynı payload güvenli modda denendiğinde psycopg2 sürücüsü girdiyi sıradan bir string değer olarak işlediği için sıfır sonuç dönmüştür. Saldırı engellenmiştir.



Şekil 11 — Güvenli mod: aynı payload 0 kayıt döndürdü, saldırı engellendi

Bu demonun en önemli mesajı, SQL injection'ın aslında bir veritabanı zafiyeti değil uygulama katmanı zafiyeti olmasıdır. Veritabanı kendisine gönderilen sorguyu doğru şekilde çalıştırmıştır; açık, kullanıcı girdisini doğrulamadan SQL'e gömen kodun kendisindedir.

3.6 Audit Log Sistemi

"Kim, ne zaman, hangi veriyi değiştirdi?" sorusuna kesin cevap verebilmek bankacılık denetiminin temel şartıdır. Bu projede PostgreSQL trigger mekanizması kullanılarak musteriler tablosundaki tüm INSERT, UPDATE ve DELETE işlemleri otomatik olarak audit_log tablosuna kaydedilmiştir. Trigger fonksiyonu PL/pgSQL ile yazılmış olup, değişikliğin öncesi ve sonrasını JSONB formatında saklar.

Test için banka_admin rolü ile bir müşterinin telefon numarası güncellenmiş ve yeni bir müşteri eklenmiştir. Audit log işlemden önce boş iken iki adet kayıt otomatik olarak oluşmuştur.

```
=== 1. Audit log boş mu? ===
 audit_sayisi
--------------
            0
(1 satır)


=== 2. banka_admin bir musteriyi guncelliyor ===
UPDATE 1

=== 3. Yeni bir musteri ekleniyor ===
INSERT 0 1

=== 4. Audit log son 3 kayit ===
 log_id | tablo_adi  | islem_tipi | kayit_id | kullanici_adi |        islem_zamani
--------+------------+------------+----------+---------------+----------------------------
      2 | musteriler | INSERT     |     1001 | banka_admin   | 2026-04-20 23:15:44.821307
      1 | musteriler | UPDATE     |        1 | banka_admin   | 2026-04-20 23:15:44.794081
(2 satır)

(venv) kerem@192 sql %
```

Şekil 12 — Trigger sonrası otomatik audit kayıtları

Sistemin gücünü gösteren en çarpıcı özellik, eski ve yeni veriyi yan yana karşılaştırabilmesidir. Aşağıdaki sorgu, UPDATE kaydında telefon numarasının ne olduğunu ve neye değiştirildiğini gösterir.

```
(venv) kerem@192 sql % echo "=== UPDATE kaydinda eski ve yeni veri karsilastirmasi ==="
psql -d finans_guvenlik_db -c "SELECT log_id, kullanici_adi, eski_veri->>'telefon' AS eski_telefon, yeni_veri->>'telefon' AS yeni_telefon FROM audit_log WHERE islem_tipi = 'UPDATE' ORDER BY log_id DESC LIMIT 1;"
=== UPDATE kaydinda eski ve yeni veri karsilastirmasi ===
 log_id | kullanici_adi | eski_telefon | yeni_telefon
--------+---------------+--------------+--------------
      1 | banka_admin   | 05320129193  | 05550000001
(1 satır)
```

Şekil 13 — Eski ve yeni telefon numarasının audit_log üzerinden karşılaştırması

Audit_log tablosuna SELECT yetkisi yalnızca banka_admin rolüne verilmiştir. Bu sayede kullanıcılar kendi aktivitelerini ne görebilir ne de silebilir. Denetim kayıtlarının bütünlüğü bu yetki tasarımı ile korunmaktadır.

4. Sonuç

Bu proje kapsamında bir veritabanının iki temel boyutu uygulamalı olarak ele alınmıştır. Performans tarafında, doğru ölçüm ve doğru indeks tasarımının ne kadar etkili olabildiği görülmüştür. 1 milyon satırlık tabloda yapılan basit bir EXPLAIN ANALYZE ile darboğaz tespit edilmiş, kısmi ve bileşik indeks tasarımı ile sorgu süresi 118.947 ms değerinden 72.055 ms değerine düşürülmüştür.

Güvenlik tarafında ise hiçbir tek başına önlemin yeterli olmadığı, gerçek koruma için katmanlı bir savunmanın şart olduğu görülmüştür. Şifreleme verinin diskten sızdırılmasını engellerken, RLS satır bazlı yetkisiz okumayı önlemekte; parameterized query uygulama katmanındaki SQL injection açığını kapatmakta; trigger tabanlı audit log ise yapılan her değişikliği değiştirilemez biçimde kayıt altına almaktadır. Bu dört katman bir araya gelince, ne yetkili bir analistin yetki dışı veri okuması ne de bir saldırganın açık metin veri çalması mümkün kalmaktadır.

Projenin geliştirici açısından en önemli kazanımı, veritabanı güvenliği ve performansının sonradan eklenebilecek özellikler değil, başlangıçtan itibaren tasarımın parçası olması gerektiğini somut deneyimle anlamak olmuştur.

Kaynaklar

[1] PostgreSQL 16 Documentation — https://www.postgresql.org/docs/16/

[2] PostgreSQL pgcrypto Module — https://www.postgresql.org/docs/16/pgcrypto.html

[3] PostgreSQL Row Security Policies — https://www.postgresql.org/docs/16/ddl-rowsecurity.html

[4] OWASP SQL Injection Prevention Cheat Sheet — https://cheatsheetseries.owasp.org/

[5] psycopg2 — PostgreSQL adapter for Python — https://www.psycopg.org/docs/